VS Code • C++17 • q1_complete.cpp • Terminal

```
$ g++ -std=c++17 -Wall -Wextra q1_complete.cpp -o q1_complete
$ ./q1_complete
OUTPUT
after call by value    : 45000
after call by reference: 40500

--- Source code ---
```

```cpp
#include <iostream>

void applyByValue(long long fee, int percent)
{
    fee -= fee * percent / 100;   // changes only the local copy
}

void applyConcession(long long& fee, int percent)
{
    // TODO 1: Reduce the caller's fee by percent using integer arithmetic (call by reference)
    fee -= fee * percent / 100;
}

int main()
{
    long long hostelFee = 45000;
    applyByValue(hostelFee, 10);
    std::cout << "after call by value    : " << hostelFee << '\n';
    applyConcession(hostelFee, 10);
    std::cout << "after call by reference: " << hostelFee << '\n';
}
```

VS Code • C++17 • q1_predict.cpp • Terminal

```
$ g++ -std=c++17 -Wall -Wextra q1_predict.cpp -o q1_predict
$ ./q1_predict
OUTPUT
1 12 13
1 100
16 13 6

--- Source code ---
```

```cpp
#include <iostream>

void change(int a, int& b, int* c)
{
    a += 10;
    b += 10;
    *c += 10;
}

int& bigger(int& p, int& q) { return p > q ? p : q; }

void show(int x, int y = 5, int z = 10) { std::cout << x + y + z << ' '; }

int main()
{
    int a = 1, b = 2, c = 3;
    change(a, b, &c);
    std::cout << a << ' ' << b << ' ' << c << '\n';

    bigger(a, c) = 100;
    std::cout << a << ' ' << c << '\n';

    show(1);
    show(1, 2);
    show(1, 2, 3);
    std::cout << '\n';
}
```

VS Code • C++17 • q2_marks.cpp • Terminal

```
$ g++ -std=c++17 -Wall -Wextra q2_marks.cpp -o q2_marks
$ ./q2_marks
OUTPUT
q2_marks.cpp:25:12: warning: reference to local variable 'localMark' returned [-Wreturn-local-addr]
Original marks: 72 88 91 65 84
After +2 moderation to highest: 72 88 93 65 84
After assigning 100 to highest: 72 88 100 65 84
Highest (const ref): 100
Before vector growth: refBefore = 85 (addr 0x56b6b72a7ec4)
After vector growth : vec[1] = 85 (new addr 0x56b6b72e7ee4), index 1 remains valid!

--- Source code ---
```

```cpp
#include <iostream>
#include <vector>

// 1. Return reference to highest mark in array
int& highestMark(int marks[], int n) {
    int maxIdx = 0;
    for (int i = 1; i < n; ++i) {
        if (marks[i] > marks[maxIdx]) maxIdx = i;
    }
    return marks[maxIdx];
}

// 2. Const reference version (read-only access)
const int& highestMarkConst(const int marks[], int n) {
    int maxIdx = 0;
    for (int i = 1; i < n; ++i) {
        if (marks[i] > marks[maxIdx]) maxIdx = i;
    }
    return marks[maxIdx];
}

// 3. Dangerous function: returns reference to local variable
int& dangerousLocalRef() {
    int localMark = 95;
    return localMark; // Compiler warns: -Wreturn-local-addr
}

// 4. Alternative: return index (safe even if vector reallocates)
int highestMarkIndex(const std::vector<int>& v) {
    int maxIdx = 0;
    for (int i = 1; i < static_cast<int>(v.size()); ++i) {
        if (v[i] > v[maxIdx]) maxIdx = i;
```

VS Code • C++17 • q3_courier.cpp • Terminal

```
$ g++ -std=c++17 -Wall -Wextra q3_courier.cpp -o q3_courier
$ ./q3_courier
OUTPUT
1) charge(2)            :   [Selected: charge(int grams, bool, bool)] -> Rs. 0.1
2) charge(2.0f)         :   [Selected: charge(double kg, bool, bool)]  -> Rs. 100
3) charge(2.5)          :   [Selected: charge(double kg, bool, bool)]  -> Rs. 125
4) charge(2.5, true)    :   [Selected: charge(double kg, bool, bool)]  -> Rs. 225
5) charge(1500, true, true):   [Selected: charge(int grams, bool, bool)] -> Rs. 215
6) charge(Parcel)       :   [Selected: charge(const Parcel&)]          -> Rs. 500

--- Source code ---
```

```cpp
#include <iostream>
#include <string>

struct Parcel {
    double weightKg;
    bool express = false;
    bool insured = false;
    std::string zone = "Domestic";
};

// Overload 1: weight in grams (int) with default arguments from the right
double charge(int grams, bool express = false, bool insured = false) {
    double kg = grams / 1000.0;
    double cost = kg * 50.0 + (express ? 100.0 : 0.0) + (insured ? 40.0 : 0.0);
    std::cout << "  [Selected: charge(int grams, bool, bool)] -> Rs. ";
    return cost;
}

// Overload 2: weight in kg (double) with default arguments from the right
double charge(double kg, bool express = false, bool insured = false) {
    double cost = kg * 50.0 + (express ? 100.0 : 0.0) + (insured ? 40.0 : 0.0);
    std::cout << "  [Selected: charge(double kg, bool, bool)]  -> Rs. ";
    return cost;
}

// Overload 3: Parcel object passed by const reference
double charge(const Parcel& p) {
    double base = p.weightKg * 50.0 + (p.express ? 100.0 : 0.0) + (p.insured ? 40.0 : 0.0);
    if (p.zone == "International") base *= 2.0;
    std::cout << "  [Selected: charge(const Parcel&)]          -> Rs. ";
    return base;
}
```

VS Code • C++17 • q4_savings.cpp • Terminal

```
$ g++ -std=c++17 -Wall -Wextra q4_savings.cpp -o q4_savings
$ ./q4_savings
OUTPUT
Mini-Statement for Account: SB-19242139 | Balance: Rs. 26000
Last 10 transaction(s) (oldest -> newest):
  1. Deposit   +1500    ...   6. Deposit   +4000
  7. Withdraw  -200     ...  10. Withdraw  -800

--- Source code ---
```

```cpp
#include <iostream>
#include <string>

class SavingsAccount {
private:
    std::string accountNumber;
    double balance;
    double transactions[10]; // positive = deposit, negative = withdrawal
    int count;               // total transactions recorded (up to 10 stored, or circular)
    int startIdx;            // circular buffer start index

    void recordCircular(double amount) {
        if (count < 10) {
            transactions[(startIdx + count) % 10] = amount;
            ++count;
        } else {
            transactions[startIdx] = amount;
            startIdx = (startIdx + 1) % 10;
        }
    }

public:
    // Defined inside the class
    SavingsAccount(const std::string& accNo, double initialBalance)
        : accountNumber(accNo), balance(initialBalance), count(0), startIdx(0) {
        for (int i = 0; i < 10; ++i) transactions[i] = 0.0;
    }

    double getBalance() const { return balance; }

```

VS Code • C++17 • q5_busticket_complete.cpp • Terminal

```
$ g++ -std=c++17 -Wall -Wextra q5_busticket_complete.cpp -o q5_busticket_complete
$ ./q5_busticket_complete
OUTPUT
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3

--- Source code ---
```

```cpp
#include <iostream>
#include <string>

class BusTicket {
    static int issued;        // shared by all BusTicket objects
    static const int CAPACITY = 3;
    int seat;
    std::string name;
public:
    explicit BusTicket(const std::string& n) : seat(0), name(n)
    {
        // TODO 1: If fewer than CAPACITY tickets have been issued, increase issued and
        // give this ticket the next seat number; otherwise leave seat as 0 (waiting list)
        if (issued < CAPACITY) {
            seat = ++issued;
        }
    }

    static int count()
    {
        // TODO 2: Return the number of seats issued so far
        return issued;
    }

    void print() const
    {
        std::cout << name << ": "
                  << (seat ? "Seat " + std::to_string(seat) : std::string("Waiting"))
                  << '\n';
    }
```

VS Code • C++17 • q6_time_complete.cpp • Terminal

```
$ g++ -std=c++17 -Wall -Wextra q6_time_complete.cpp -o q6_time_complete
$ ./q6_time_complete
OUTPUT
205 false
Either side of noon (11:50 vs 12:15): 25 mins, earlier=true
Same time twice (12:00 vs 12:00)    : 0 mins, earlier=false
[Benchmark 10M calls] -O0 noinline: 68.85 ms | -O0 inline: 69.30 ms
[Benchmark 10M calls] -O2 noinline: 28.56 ms | -O2 inline: 20.51 ms

--- Source code ---
```

```cpp
#include <iostream>

class Time {
    int h, m;
public:
    Time(int hh, int mm) : h(hh), m(mm) {}
    friend int minutesBetween(const Time& a, const Time& b);
    friend bool earlier(const Time& a, const Time& b);
};

int minutesBetween(const Time& a, const Time& b)
{
    // TODO 1: Using the private members h and m, return the absolute difference between
    // the two times in minutes
    int t1 = a.h * 60 + a.m;
    int t2 = b.h * 60 + b.m;
    return (t1 > t2) ? (t1 - t2) : (t2 - t1);
}

bool earlier(const Time& a, const Time& b)
{
    // TODO 2: Return true if a is earlier in the day than b
    return (a.h < b.h) || (a.h == b.h && a.m < b.m);
}

int main()
```